Sisteme Distribuite

Laboratorul 2

Comunicația inter-proces

Andrei Pătrașcu

Răzvan Anghel

1 Introducere în Limbajul Go - continuare

Funcții

În limbajul Go, funcțiile se definesc folosind keyword-ul `func`, urmat de numele funcției, lista de parametrii (fiecare dintre ei trebuie să aibă specificat tipul) și tipul de date returnat. Instrucțiunea `return` este opțională, lipsa ei indică o funcție de tip void.

```
// Sintaxa:
func nume_functie( parametru_1 tip_parametru_1, ... ) tip_de_date_returnat {
    // acolada trebuie sa fie pusa obligatoriu pe acelasi rand cu antetul
    // corpul functiei
    return // opional
}
```

O funcție în Go poate returna mai multe valori, separate prin virgulă. Acest lucru se specifică în antetul funcției, înainte de acoladă, tipurile datelor returnate trecându-se între paranteze.

```
func intDivision(a int, b int) (int, int) {
    return a / b, a % b
}
```

Go permite și declararea unor variabile în momentul declarării tipului returnat. Ele vor fi inițializate cu valorile nule de tip.

Conform convenției de tratare a erorilor din Go (Error Handling Idiom), o funcție care poate declanșa o eroare trebuie să returneze ca ultim tip de date returnat o eroare, adică `(val, error)`. Cu alte cuvinte, în Go nu există excepții, iar gestionarea erorii se face, de regulă, din funcția apelantă. Astfel, funcția de mai sus devine [7]:

```
func intDivision(a int, b int) (int, int, err error) {

    if( b == 0 ){
        err = errors.New("Cannot divide by zero!")
        return -1, -1, err
    }

    return a / b, a % b, err
}
```

Defer

Defer este un keyword plasat înaintea unei apel de funcție, din interiorul unei funcții. Apelul va fi plasat pe stiva funcției apelante și va fi executat imediat ce ea își termină execuția normală (și,

eventual returnează un rezultat) dar înainte ca ea să fie scoasă de pe stivă. Cu alte cuvinte, plasarea lui *defer* înaintea apelării unei alte funcții are ca efect execuția apelului înainte ca funcția să se termine. Prin mecanismul de plasare pe stivă a apelului, acesta va fie executat indiferent dacă funcția încheie execuția normal sau întâmpină o eroare pe parcurs. *Defer* devine util în momentul în care vrem ca resursele utilizate să fie eliberate chiar și în caz de eroare, respectiv avem nevoie ca anumite operații să se finalizeze odată cu funcția, de exemplu la *waitgroups*.

```
    func main() {

        // Aceste doua apeluri vor fi puse pe stiva functiei main
        defer fmt.Println("1")
        defer fmt.Println("2")
        // Deoarece vor fi scoase de pe stiva în ordine inversa, se va afisa: 2 1

        // Variabilele folosite de defer vor avea valorile de la momentul
    ăevaluarii
        i := 0
        defer fmt.Println(i) // Se va afisa: 0
        i++
}
```

String-uri și rune

Go folosește sistemul de codare UTF-8 pentru a reprezenta caracterele din string-uri. El suportă orice caracter UNICODE (diacritice, caractere japoneze și arabe, emoji-uri etc.) Standardul UTF-8 este o codare pe un număr variabil de bytes (între 1 și 4), fiind compatibil cu ASCII, caracterele acestea având și cea mai mică reprezentare, deoarece sunt folosite cel mai des, în aproape toate limbile. Pentru mai multe detalii despre modul de funcționare și conversie al UTF-8 în caractere UNICODE, vă recomandăm [2, 7].

Datorită codării pe lungime variabilă, un string în Go are la bază un `byte` array. Din această cauză apelarea funcției `len()` pentru un `string` va avea ca rezultat numărul de bytes pe care îl ocupă acel string, nu numărul efectiv de caractere al său:

```
    // Acesta este un string compus doar din caractere ASCII
    text_ascii := "This"
    // Lungimea sa va fi egala cu numarul de caractere
    fmt.Println(len(text_ascii))  // => 4

    // Daca exista si caractere non-ASCII, nu mai e cazul
    text_unicode := "Atât"
    fmt.Println(len(text_unicode))  // => 5
    // Output-ul este 5 deoarece caracterul non-ascii "â" e reprezentat pe 2 bytes
     în UTF-8
```

Din acest motiv, când trebuie să lucrăm cu string-uri, de cele mai multe ori, preferăm transformarea lor într-un **array de rune**. O runa este un alias pentru `int32` și gestionează conversia de la caractere UTF-8 la codepoint-uri UNICODE. Astfel, prin conversia unui string la un array de rune, obținem un array de codepoint-uri UNICODE, adică putem afla lungimea corectă a string-ului inițial:

```
    text_unicode := "Atât"
    rune_conversion := []rune(text_unicode)
    fmt.Println(len(rune_conversion))  // => 4

    // Este de preferat sa folosim alternativa utf8.RuneCountInString, care face
    exact acelasi lucru, într-o singura linie de cod:
    fmt.Println(utf8.RuneCountInString(text_unicode)) // => 4
```

În Go, *string-urile* sunt **imutabile**: nu se pot schimba litere individuale din array-ul de bytes care stă la baza string-ului, dar se poate schimba referința string-ului către altul. Concatenarea se

poate face cu operatorul "+", dar este recomandată folosirea unui string builder din pachetul `strings`. Pentru alte funcții din acest pachet, vă lăsăm la dispoziție documentația oficială în [5].

Structuri

Structurile sunt tipuri speciale de date, definite de către programatori. Funcționează similar cu clasele din orice alt limbaj. La declararea unei variabile de tipul unui struct, câmpurile ei vor fi inițializate cu valoarea nulă de tip. Structurile se declară și inițializează în următoarele feluri:

```
// Sintaxa declarare:
type numeTip struct {
    campuri...
}

// Exemplu:
type Student struct {
    name string
    age int8
}

// Exemple de atribuiri:
var student_A Student = Student{name:"Alex", age:19}
var student_B Student = Student{"Horia", 22}
```

Atât câmpurile cât și structurile pot fi declarate atât cu literă mare cât și cu literă mică, diferența fiind vizibilitatea în exteriorul pachetului curent. Câmpurile și struct-urile scrise cu litere mari sunt exportate și, prin urmare, sunt vizibile și în alte pachete. Aceste reguli se aplică și funcțiilor, metodelor, variabilelor globale și constantelor.

Structurile admit și metode, care se declară ca niște funcții, în exteriorul struct-ului după cum se poate observa în exemplul de mai jos. Practic, o funcție devine o metodă (are acces la câmpurile unei instanțe a struct-ului) prin faptul că are un receiver de tipul struct-ului, între keyword-ul `func` și numele funcției.

```
// Sintaxa pentru metode:
func (nume\_variabila nume\_tip\_struct) nume\_functie( parametrii ) valoare\
_returnata {}

// Exemplu de metoda:
func (s Student) can_receive_disounts() bool {

    if s.age >= 30 {
        return false
    }

    return true
}
```

Pointeri

Pointerii sunt variabile de 32 sau 64 biți (în funcție de arhitectura pe care se compilează) care stochează adrese de memorie. Ei sunt inițializați implicit cu `nil`, dar se pot inițializa cu funcția `new(tip_de_date)` care va aloca spațiu liber și va scrie în pointer locația de memorie unde s-a alocat acest spațiu liber. Se poate folosi operatorul * pentru a declara și dereferența pointeri și *&* pentru a accesa adresa unei variabile:

```
var i int
```

```go
// Declarara unui pointer
var p *int = new(int)

p = &i
*p = 10  // => i = 10
```

Slice-urile, array-urile, map-urile și canalele **sunt, de fapt, pointeri**. Funcțiile din Go *transmit parametrii implicit prin copiere* a valorii. Pentru transmiterea prin **referință**, parametrul trebuie schimbat într-un pointer.

2 Comunicație inter-proces în limbajul Go

Procese

Un proces este un program aflat în execuție. El este organizat în mai multe secțiuni din memorie după cum urmează:

- text — conține codul binar al programului, alcătuit din instrucțiunile pe care le execută procesorul. Practic, secțiunea text e constituită de executabilul (binarul) stocat pe disk înainte de pornirea execuției.
- data — stochează variabilele statice și globale pe care le folosește programul.
- stack — zonă fixă de memorie care se poate extinde până la o mărime maximă. Aici se stochează informații asociate funcțiilor (parametrii, variabile locale și adrese de retur) care sunt folosite pentru a gestiona trecerea de la o funcție la alta în timpul execuției.
- heap — zonă dinamică de memorie (accesată prin *new* sau *malloc*), mult mai mare și mai lentă decât zona stack, unde se stochează alte tipuri de obiecte, de regulă, de dimensiuni mai mari.

Sistemul de operare asociază fiecărui proces o structură de date numită *Process Control Block* care reține: id-ul procesului (pid), starea procesului (*ready, waiting, running, terminated*), program counter (instruction pointer), copii ale valorilor regiștrilor de la ultimul ciclu de execuție pe CPU, informații pentru scheduling (prioritate, durată estimată de rulare etc.), statistici din timpul rulării, dar și informații despre memorie și I/O [8].

Modalități de comunicare inter-proces

În sistemele single-core, sistemul de operare poate realiza context switch-uri extrem de rapide, pentru a simula executarea mai multor programe în același timp. În acest fel, utilizatorii aveau impresia că mai multe programe rulează în același timp. Datorită faptului că fiecare proces are propria zonă de memorie și a simulării execuției paralele, procesele au trebuit să comunice între ele sau să gestioneze resurse comune. Astfel, au apărut mecanisme de sincronizare (mutex-uri, semafoare, bariere etc.), de planificare și de comunicare interproces, realizată prin *memorie partajată* sau *message passing*. Implementări ale acestui mecanism sunt canalele (sau pipe-urile, pe care le vom prezenta în acest laborator), cozile de mesaje, socket-urile (comunicare prin intermediul unor protocoale la nivel de rețea) sau Remote Procedure Call (numită și RPC, fiind un protocol care permite execuția unei funcții pe un alt calculator) [8].

Thread-uri

Thread-urile, pe de altă parte, sunt cele mai compacte unități de execuție care pot fi rulate de un procesor, iar structura lor de date asociată thread control block reține: un thread id, un instruction pointer, o zonă proprie de stack și copii ale regiștrilor procesorului. Mai multe thread-uri în cadrul aceluiași proces au aceeași zonă de heap și aceleași resurse de I/O. Sincronizarea între thread-uri rămâne o problemă importantă pentru programele multi-thread, iar mecanismele menționate anterior se pot folosi și pentru gestionarea execuției thread-urilor [8].

Avantajul principal al thread-urilor îl constituie memoria partajată, care este accesată extrem de rapid, comparativ cu procesele care au alocate secțiuni distincte de memorie. În funcție de sarcinile

pe care le execută fiecare thread, mecanismele de sincronizare nu majorează substanțial timpul total de execuție al programului, ba chiar este redus considerabil pe procesoarele multicore, arhitectură întâlnită în majoritatea calculatoarelor de astăzi.

Thread-urile se împart în **kernel threads** (gestionate direct de nucleul sistemului de operare) și **user threads** (gestionate în user space de runtime-uri precum JVM sau Go). Kernel threads au un consum mare de RAM și un overhead ridicat la context switching din cauza tranzițiilor în kernel space, care se fac prin syscalls. În schimb, user threads sunt mult mai lightweight, deoarece context switching-ul și scheduling-ul lor au loc direct în memorie, fără intervenția OS. De exemplu, un milion de user threads ocupă 2GB RAM, în timp ce în kernel ar ocupa 1TB RAM.

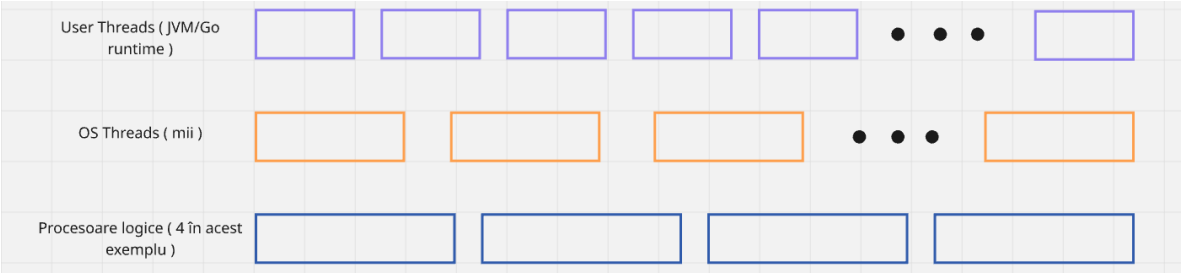


Figura 1: User threads, kernel threads și procesoarele logice

Paralelism și concurență

Este important să facem distincția dintre aceste două concepte. Ele sunt similare, deoarece se referă la execuția simultană, într-o formă sau alta, a mai multor task-uri. Paralelismul presupune rularea independentă a fiecărui task folosind nuclee diferite ale procesorului, iar concurența semnifică mutarea succesivă de la un task la altul până când toate sunt gata. Vă mai puteți raporta și la citatul lui Rob Pike, unul dintre oamenii din spatele acestui limbaj: *„Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once”* [3].

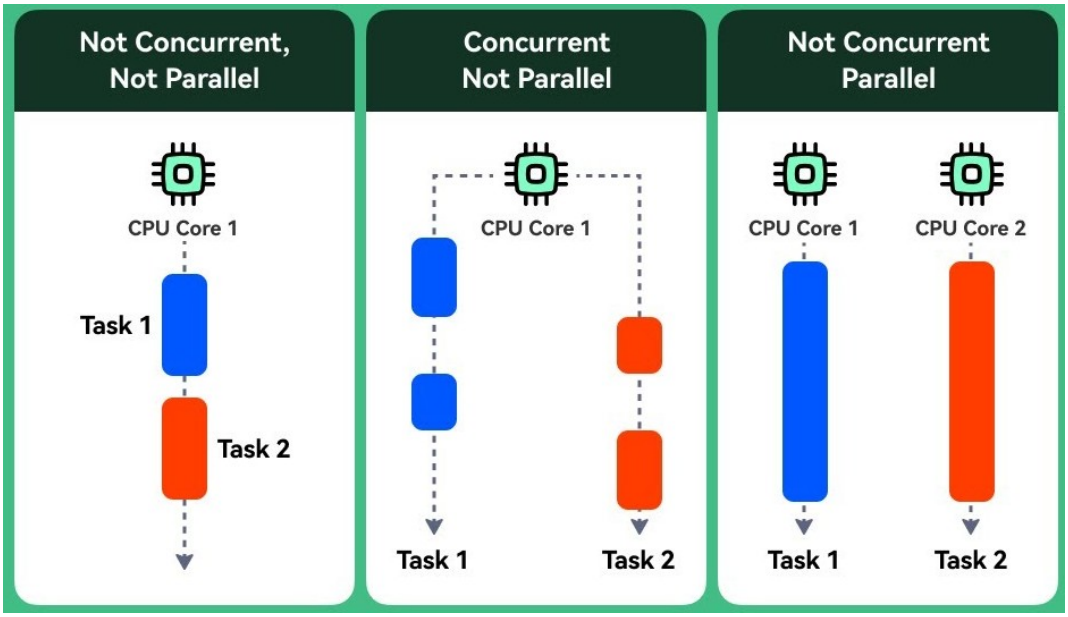


Figura 2: Diferența dintre paralelism și concurență [9]

O analogie bună ar fi niște „task-uri” care trebuie realizate într-o cameră de cămin (spălat rufe, aspirat, spălat vase etc.) O singură persoană le poate executa secvențial (adică pe rând) sau concurent

(spală o parte din vase, apoi aspiră jumătate de cameră, apoi sortează o parte din rufe și schimbă „task-ul" curent, până când le termină pe toate). În acest mod, o persoană se ocupă practic de toate „task-urile" în același timp, adică le începe pe toate, urmând să le termine la un moment dat. Acest lucru poate fi avantajos deoarece, în funcție de tipul „task-ului" poate duce la scăderea timpului total de lucru. Spre exemplu, în modelul concurent, persoana poate aspira și spăla vasele în timp ce mașina de spălat funcționează, urmând să le întindă după ce acestea se termină.

În modelul paralel, task-urile sunt executate simultan de mai multe persoane, ceea ce duce la diminuarea maximă a timpului total de finalizare a sarcinilor. Acest tip de paralelizare în execuția thread-urilor se realizează numai pe arhitecturi de tip multicore. În cazul în care numărul de thread-uri depășește numărul de nuclee ale procesorului, doar o submulțime a acestora se vor executa în paralel, restul desfășurându-se concurent.

Goroutines

După cum am scris mai sus, concurența se poate realiza prin execuția paralelă a task-urilor pe mai multe CPU-uri sau prin alternarea task-urilor pe același CPU. *Goroutines* sunt fire de execuție gestionate de runtime-ul Go. Ele sunt mai eficiente decât fire de execuție gestionate de sistemul de operare, deoarece necesită mai puțină memorie, iar timpul de pornire este mult mai mic. Fiecare program Go are o **goroutine principală**, din care pornesc alte goroutines. Când aceasta se oprește, se opresc și cele care au pornit din ea. Mai concret, o goroutine poate fi orice funcție, pe care programatorul o pornește pe alt fir de execuție folosind keyword-ul **go** în fața numelui funcției [7, 4, 1]:

```
func sum(v []int) {

    s := 0

    for i := range v {
        s += v[i]
    }

    fmt.Printf("Suma afisata din interiorul acestei goroutine este: %v\n", s)
}

func main() {

    intSlice := []int{4, 6, 8, 16}

    // Am rulat functia sum ca o goroutine
    go sum(intSlice) // ruleaza concurent cu goroutina main

    fmt.Println("Text afisat din goroutina main")
}
```

Se poate observa că suma slice-ului nu este afișată deoarece goroutina `main` se termină înaintea goroutinii `sum`. Pentru a evita astfel de comportamente impredictibile, se folosesc instrumente de sincronizare din pachetul sync [6]. Cele mai utile sunt:

- **Mutex**: Permite accesul unei singure goroutine asupra unei zone critice. Se folosește în scenarii în care predomină operațiile de scriere. Conține metodele `Lock()` și `Unlock()`.
- **RWMutex**: Permite mai multor goroutine să citească zona critică, dar numai uneia să o acceseze pentru scriere. Se folosește când predomină operațiile de citire. Față de un Mutex simplu, mai are și metodele `RLock()` și `RUnlock()`.
- **WaitGroup**: Este o barieră implementată cu un semafor, folosită pentru a bloca execuția până când toate goroutinele secundare se termină. Folosește medotele `Add(delta int)`, `Done()`, `Wait()` și `Go(f func())`

Un goroutine poate fi definit și folosind o funcție anonimă. Rescriind codul de mai sus cu aceste concepte, obținem:

```
    func sum(v []int, _wg *sync.WaitGroup) {

     s := 0

     for i := range v {
         s += v[i]
     }

     fmt.Printf("Suma afisata din interiorul acestei goroutine este: %v\n", s)
     _wg.Done()
}

func main() {

     intSlice_1 := []int{4, 6, 8, 16}
     intSlice_2 := []int{5, 10, 15, 20}

     // Initializam un obiect de tip sync.WaitGroup
     wg := sync.WaitGroup{}

     go sum(intSlice_1, &wg)
     wg.Add(1)  // Adaugam un thread in WaitGroup

     // Goroutina anonima definita in-place si apelata pentru intSlice_2
     // Va calcula minimul slice-ului si il va afisa
     go func(v []int) {

         // Functia anonima are acces la variabilele declarate in functia din care
     este apelata
         defer wg.Done()   // se poate folosi si defer, dupa preferinta

         min := v[0]
         for i := range v {
             if v[i] < min {
                 min = v[i]
             }
         }

         fmt.Printf("Minimul calculat din goroutina anonima este: %v\n", min)

     }(intSlice_2)
     wg.Add(1)

     // Metoda Go, gestioneaza automat scoaterea si introducerea în wait group
     wg.Go(func() {
                 fmt.Println("Folosim metoda Go...în Go")
         })

     // Executia thread-ului principal se opreste pana cand toate thread-urile
     secundare ajung aici
     wg.Wait()
     fmt.Println("Text afisat din goroutina main")
}
```

Ordinea de finalizare a execuției pentru cele două goroutine este impredictibilă, dar ele se vor finaliza înaintea funcției principale `main`, datorită barierei.

Canale

Canalele sunt **pipe-uri de comunicație bidirecționale între goroutine** care se execută concurent. Crearea canalelor se poate face folosind, din nou, funcția `make()`, cu parametrul `chan tip_de_date`. Datele se pot introduce și elimina dintr-un canal folosind operatorul `<-`, în funcție de unde este plasat canalul în raport cu operatorul: la stânga înseamnă că se introduc data în canal, iar la dreapta înseamnă că se extrag date din el. Un exemplu interesant este mai jos:

```
// Exemplu initializare canal de int-uri:
var ch = make(chan int)

ch <- 1  // Introducere de date in canal
var i = <- ch  // Extragere de date din canal
```

Dacă rulăm, runtime-ul Go va opri execuția cu eroarea: `fatal error: all goroutines are asleep - deadlock!`. De ce?

Canalul declarat mai sus este **unbuffered**, adică nu are o limită de variabile de tipul canalului care să poată fi scrise în el. Mai concret, se pot pune în canal oricâte numere întregi vrem. Însă, **când se scrie într-un unbuffered channel, execuția se blochează până când alt goroutine va citi din el**. Cu alte cuvinte, un goroutine nu va scrie într-un canal, până când alt goroutine nu va fi gata să citească din el. Astfel, scrierea și citirea se vor face sincron și thread-safe, fără a fi necesare mecanisme suplimentare de sincronizare precum mutex-uri, ceea ce face canalele perfecte pentru un mediu distribuit.

În cazul de mai sus, go routine-ul care execută funcția `main` va aștepta ca alt goroutine să citească din canalul `ch`, ceea ce nu se întâmplă, de unde și eroarea de deadlock.

Canale cu buffer

Canalele cu buffer sunt canale care au specificate și o capacitate, ca parametru în funcția `make`. Ele permit un număr de operații de scriere în canal, fără o operație de extragere echivalentă.

Canalele cu buffer sunt pipe-uri de comunicație care pot stoca într-un buffer un anumit număr de elemente. Acest număr se numește **capacitate** și poate fi specificat ca parametru în funcția `make`. Introducerea de valori în canal este blocată după ce buffer-ul se umple. Extragerea valorilor din canal nu se mai poate realiza când buffer-ul s-a golit [4]. Un canal fără buffer are capacitatea 0.

```
// Exemplu declarare canal cu buffer
ch := make(chan string, 1)

ch <- "Sisteme" // se poate scrie un singur mesaj in canal
ch <- "Distribuite" // aici se depaseste capacitatea canalului => deadlock

// Corect este sa extragem un mesaj din buffer înainte de o noua scriere
fmt.Println(<-ch)
ch <- "Distribuite"
```

Send-receive channels

Canalele sunt **implicit bidirecționale**. Câteodată este util ca ele să fie restrânse ca **send-only** sau **receive-only**, mai ales când sunt parametrii ai unor funcții. Acest lucru se poate face prin **plasarea operatorului `<-`** înaintea sau după keyword-ul **chan**.

```
// receive-only channel
func receive(ch <-chan string) string {
    return <-ch
}
```

Close

Funcția *close* primește ca parametru un canal și este folosită pentru a semnala faptul că printr-un canal nu vor mai fi trimise informații. *Close* ar trebui folosită doar de către goroutine-ul din care se trimit informații, nu și de cele care le primesc. Este foarte util să folosim *close* când există un singur sender și mai mulți receiveri și când fiecare dintre ei trebuie să știe că în canal nu se vor mai scrie informații. Putem verifica dacă un canal a fost închis asemănător cu verificarea existenței unei chei într-un map: `message, ok := <-ch`. În exemplul de mai jos se poate vedea și cum verificăm dacă un canal a fost închis [4]:

```
ch := make(chan string, 1)
go func() {
    ch <- "UNIBUC"
    close(ch)
}()

// Desi canalul a fost închis, înca exista informatie in el => receive cu
succes
message, ok := <-ch
fmt.Printf("S-a primit informatie: %t\n", ok)    // => true
fmt.Printf("Mesajul primit este: %s\n", message) // => UNIBUC

// Aici nu se mai face receive, iar message va lua valoarea nula de tip:
message, ok = <-ch
fmt.Printf("S-a primit informatie: %t\n", ok)    // => false
fmt.Printf("Mesajul primit este: %s\n", message) // => "" (empty string)
```

Range

Se poate utiliza sintaxa `range` pentru a aștepta și prelua continuu informații dintr-un canal. Folosirea `range` este blocking - execuția se blochează până când sunt scrise infirmații în canal. Singura modalitate de ieșire din buclă este închiderea canalului de către sender [4, 7].

```
ch := make(chan string, 2)
ch <- "Strada"
ch <- "Academiei"
close(ch)

for elem := range ch {
    fmt.Println(elem)
}
```

Select

Intuitiv, instrucțiunea *select* funcționează ca un *switch* pentru canale. Mai concret, select permite unui goroutine să asculte mai multe canale simultan și să execute blocuri de cod diferite, în funcție de canalul din care primește informație. *Select* ***blochează execuția*** până când un alt goroutine va scrie într-unul dintre canale. Dacă mai multe canale primesc informații simultan, *select* alege un bloc de cod în mod aleator pe care să îl execute.

Totodată, mai există și cazul *default*, care trnaformă *select* într-o instrucțiune **non-blocking**, cazul *default* executându-se imediat, dacă nu există un alt canal nu a primit informații. Folosirea unui *select* este indicată în momentul în care programul trebuie să aștepte executarea unor operațiuni, dar continuă execuția în mod normal, după un anumit timp, dacă nu primește răspunsuri:

```
ch1 := make(chan string)
ch2 := make(chan string)
```

```
// Simulam un serviciu
go func() {
    time.Sleep(100 * time.Millisecond)
    ch1 <- "The service sent a response"
}()

select {
case msg := <-ch1:
    fmt.Println("Received:", msg)
case msg := <-ch2:
    fmt.Println("Received:", msg)
// time.After() returneaza un obiect de tipul time.Time
// Sintaxa de mai jos creeaza un <-chan time.Time pe care îl asculta în select
// În acest canal va fi scris un obiect daca timp de 200 ms nu se întampla
nimic
case <-time.After(200 * time.Millisecond):
    fmt.Println("Operation timed out")
}
```

Bibliografie

[1] Ankita Saini. Goroutines - Concurrency in Golang. https://www.geeksforgeeks.org/go-language/goroutines-concurrency-in-golang/, 2026. [Ultima accesare: Septembrie 2026].

[2] Autori Wikipedia. Titlul articolului — UTF-8. https://en.wikipedia.org/wiki/UTF-8, 2026. [Ultima accesare: August 2026].

[3] Google și comunitatea Go. Concurrency is not parallelism. https://go.dev/blog/waza-talk, 2013. [Ultima accesare: Septembrie 2026].

[4] Google și comunitatea Go. A Tour of Go. https://go.dev/tour/concurrency, 2026. [Ultima accesare: Septembrie 2026].

[5] Google și comunitatea Go. Strings Package. https://pkg.go.dev/strings, 2026. [Ultima accesare: Septembrie 2026].

[6] Google și comunitatea Go. Sync Package. https://pkg.go.dev/sync, 2026. [Ultima accesare: Septembrie 2026].

[7] Alex Mux. Learn go fast: Full tutorial. https://youtu.be/8uiZC0l4Ajw?si=TRuoZlc3mv-HMzIO, 2023.

[8] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley & Sons, 10th edition, 2018.

[9] ByteByteGo (Alex Xu și Sahn Lam). Concurrency vs parallelism! https://www.youtube.com/watch?v=RlM9AfWf1WU, 2024.